Subclass Sandbox

Intent

Define behavior in a subclass using a set of operations provided by its base class.

Motivation

Every kid has dreamed of being a superhero, but unfortunately, cosmic rays are in short supply here on Earth. Games that let you pretend to be a superhero are the closest approximation. Because our game designers have never learned to say, "no", *our* superhero game aims to feature dozens, if not hundreds, of different superpowers that heroes may choose from.

Our plan is that we'll have a `Superpower` base class. Then, we'll have a derived class that implements each superpower. We'll divvy up the design doc among our team of programmers and get coding. When we're done, we'll have a hundred superpower classes.

> When you find yourself with a *lot* of subclasses, like in this example, that often means a data-driven approach is better. Instead of lots of *code* for defining different powers, try finding a way to define that behavior in *data* instead.
>
> Patterns like Type Object →, Bytecode →, and Interpreter GoF can all help.

We want to immerse our players in a world teeming with variety. Whatever power they dreamed up when they were a kid, we want in our game. That means these superpower subclasses will be able to do just about everything: play sounds, spawn visual effects, interact with AI, create and destroy other game entities, and mess with physics. There's no corner of the codebase that they won't touch.

Let's say we unleash our team and get them writing superpower classes. What's going to happen?

- *There will be lots of redundant code.* While the different powers will be wildly varied, we can still expect plenty of overlap. Many of them will spawn visual effects and play sounds in the same way. A freeze ray, heat ray, and Dijon mustard ray are all pretty similar when you get down to it. If the people implementing those don't coordinate, there's going to be a lot of duplicate code and effort.

- *Every part of the game engine will get coupled to these classes.* Without knowing better, people will write code that calls into subsystems that were never meant to be tied directly to the superpower classes. If our renderer is

organized into several nice neat layers, only one of which is intended to be used by code outside of the graphics engine, we can bet that we'll end up with superpower code that pokes into every one of them.

- *When these outside systems need to change, odds are good some random superpower code will get broken.* Once we have different superpower classes coupling themselves to various and sundry parts of the game engine, it's inevitable that changes to those systems will impact the power classes. That's no fun because your graphics, audio, and UI programmers probably don't want to also have to be gameplay programmers *too*.

- *It's hard to define invariants that all superpowers obey.* Let's say we want to make sure that all audio played by our powers gets properly queued and prioritized. There's no easy way to do that if our hundred classes are all directly calling into the sound engine on their own.

What we want is to give each of the gameplay programmers who is implementing a superpower a set of primitives they can play with. You want your power to play a sound? Here's your `playSound()` function. You want particles? Here's `spawnParticles()`. We'll make sure these operations cover everything you need to do so that you don't need to `#include` random headers and nose your way into the rest of the codebase.

We do this by making these operations *protected methods of the* `Superpower` *base class*. Putting them in the base class gives every power subclass direct, easy access to the methods. Making them protected (and likely non-virtual) communicates that they exist specifically to be *called* by subclasses.

Once we have these toys to play with, we need a place to use them. For that, we'll define a *sandbox method*, an abstract protected method that subclasses must implement. Given those, to implement a new kind of power, you:

1. Create a new class that inherits from `Superpower`.

2. Override `activate()`, the sandbox method.

3. Implement the body of that by calling the protected methods that `Superpower` provides.

We can fix our redundant code problem now by making those provided operations as high-level as possible. When we see code that's duplicated between lots of the subclasses, we can always roll it up into `Superpower` as a new operation that they can all use.

We've addressed our coupling problem by constraining the coupling to one place. `Superpower` itself will end up coupled to the different game systems, but our hundred derived classes will not. Instead, they are *only* coupled to their base class. When one of those game systems changes, modification to `Superpower` may be necessary, but dozens of subclasses shouldn't have to be touched.

This pattern leads to an architecture where you have a shallow but wide class hierarchy. Your inheritance chains aren't *deep*, but there are a *lot* of classes that hang off `Superpower`. By having a single class with a lot of direct subclasses, we have a point of leverage in our codebase. Time and love that we put into `Superpower` can benefit a wide set of classes in the game.

> Lately, you find a lot of people criticizing inheritance in object-oriented languages. Inheritance *is* problematic—there's really no deeper coupling in a codebase than the

one between a base class and its subclass—but I find *wide* inheritance trees to be easier to work with than *deep* ones.

The Pattern

A **base class** defines an abstract **sandbox method** and several **provided operations**. Marking them protected makes it clear that they are for use by derived classes. Each derived **sandboxed subclass** implements the sandbox method using the provided operations.

When to Use It

The Subclass Sandbox pattern is a very simple, common pattern lurking in lots of codebases, even outside of games. If you have a non-virtual protected method laying around, you're probably already using something like this. Subclass Sandbox is a good fit when:

- You have a base class with a number of derived classes.
- The base class is able to provide all of the operations that a derived class may need to perform.
- There is behavioral overlap in the subclasses and you want to make it easier to share code between them.
- You want to minimize coupling between those derived classes and the rest of the program.

Keep in Mind

"Inheritance" is a bad word in many programming circles these days, and one reason is that base classes tend to accrete more and more code. This pattern is particularly susceptible to that.

Since subclasses go through their base class to reach the rest of the game, the base class ends up coupled to every system *any* derived class needs to talk to. Of course, the subclasses are also intimately tied to their base class. That spiderweb of coupling makes it very hard to change the base class without breaking something—you've got the brittle base class problem.

The flip side of the coin is that since most of your coupling has been pushed up to the base class, the derived classes are now much more cleanly separated from the rest of the world. Ideally, most of your behavior will be in those subclasses. That means much of your codebase is isolated and easier to maintain.

Still, if you find this pattern is turning your base class into a giant bowl of code stew, consider pulling some of the provided operations out into separate classes that the base class can dole out responsibility to. The Component→ pattern can help here.

Sample Code

Because this is such a simple pattern, there isn't much to the sample code. That doesn't mean it isn't useful—the pattern is about the *intent*, not the complexity of its implementation.

We'll start with our `Superpower` base class:

```
class Superpower
{
public:
  virtual ~Superpower() {}

protected:
  virtual void activate() = 0;

  void move(double x, double y, double z)
  {
    // Code here...
  }

  void playSound(SoundId sound, double volume)
  {
    // Code here...
  }

  void spawnParticles(ParticleType type, int count)
  {
    // Code here...
  }
};
```

The `activate()` method is the sandbox method. Since it is virtual and abstract, subclasses *must* override it. This makes it clear to someone creating a power subclass where their work has to go.

The other protected methods, `move()`, `playSound()`, and `spawnParticles()`, are the provided operations. These are what the subclasses will call in their implementation of `activate()`.

We didn't implement the provided operations in this example, but an actual game would have real code there. Those methods are where `Superpower` gets coupled to other systems in the game — `move()` may call into physics code, `playSound()` will talk to the audio engine, etc. Since this is all in the *implementation* of the base class, it keeps that coupling encapsulated within `Superpower` itself.

OK, now let's get our radioactive spiders out and create a power. Here's one:

```
class SkyLaunch : public Superpower
{
protected:
  virtual void activate()
  {
    // Spring into the air.
    playSound(SOUND_SPROING, 1.0f);
    spawnParticles(PARTICLE_DUST, 10);
    move(0, 0, 20);
  }
};
```

OK, maybe being able to *jump* isn't all that *super*, but I'm trying to keep things basic here.

This power springs the superhero into the air, playing an appropriate sound and kicking up a little cloud of dust. If all of the superpowers were this simple — just a combination of sound, particle effect, and motion — then we wouldn't need this pattern at all. Instead, `Superpower` could have a baked-in implementation of `activate()` that accesses fields for the sound ID, particle type, and movement. But that only works when every power essentially works the same way with only some differences in data. Let's elaborate on it a bit:

```
class Superpower
{
protected:
  double getHeroX()
  {
    // Code here...
  }

  double getHeroY()
  {
    // Code here...
  }

  double getHeroZ()
  {
    // Code here...
  }

  // Existing stuff...
};
```

Here, we've added a couple of methods to get the hero's position. Our `SkyLaunch` subclass can now use those:

```
class SkyLaunch : public Superpower
{
protected:
  virtual void activate()
  {
    if (getHeroZ() == 0)
    {
      // On the ground, so spring into the air.
      playSound(SOUND_SPROING, 1.0f);
      spawnParticles(PARTICLE_DUST, 10);
      move(0, 0, 20);
    }
    else if (getHeroZ() < 10.0f)
    {
      // Near the ground, so do a double jump.
      playSound(SOUND_SWOOP, 1.0f);
      move(0, 0, getHeroZ() - 20);
    }
    else
    {
      // Way up in the air, so do a dive attack.
      playSound(SOUND_DIVE, 0.7f);
      spawnParticles(PARTICLE_SPARKLES, 1);
      move(0, 0, -getHeroZ());
    }
  }
};
```

Since we have access to some state, now our sandbox method can do actual, interesting control flow. Here, it's still just a couple of simple `if` statements, but you can do anything you want. By having the sandbox method be an actual full-fledged method that contains arbitrary code, the sky's the limit.

> Earlier, I suggested a data-driven approach for powers. This is one reason why you may decide *not* to do that. If your behavior is complex and imperative, it is more difficult to define in data.

Design Decisions

As you can see, Subclass Sandbox is a fairly "soft" pattern. It describes a basic idea, but it doesn't have a lot of detailed mechanics. That means you'll be making some interesting choices each time you apply it. Here are some questions to consider.

What operations should be provided?

This is the biggest question. It deeply affects how this pattern feels and how well it works. At the minimal end of the spectrum, the base class doesn't provide *any* operations. It just has a sandbox method. To implement it, you'll have to call into systems outside of the base class. If you take that angle, it's probably not even fair to say you're using this pattern.

On the other end of the spectrum, the base class provides *every* operation that a subclass may need. Subclasses are *only* coupled to the base class and don't call into any outside systems whatsoever.

> Concretely, this means each source file for a subclass would only need a single `#include`—the one for its base class.

Between these two points, there's a wide middle ground where some operations are provided by the base class and others are accessed directly from the outside system that defines it. The more operations you provide, the less coupled subclasses are to outside systems, but the *more* coupled the base class is. It removes coupling from the derived classes, but it does so by pushing that up to the base class itself.

That's a win if you have a bunch of derived classes that were all coupled to some outside system. By moving the coupling up into a provided operation, you've centralized it into one place: the base class. But the more you do this, the bigger and harder to maintain that one class becomes.

So where should you draw the line? Here are a few rules of thumb:

- If a provided operation is only used by one or a few subclasses, you don't get a lot of bang for your buck. You're adding complexity to the base class, which affects everyone, but only a couple of classes benefit.

  This may be worth it to make the operation consistent with other provided operations, or it may be simpler and cleaner to let those special case subclasses call out to the external systems directly.

- When you call a method in some other corner of the game, it's less intrusive if that method doesn't modify any state. It still creates a coupling, but it's a "safe" coupling because it can't break anything in the game.

  > "Safe" is in quotes here because technically, even just accessing data can cause problems. If your game is multi-threaded, you could read something at the same time that it's being modified. If you aren't careful, you could end up with bogus data.
  >
  > Another nasty case is if your game state is strictly deterministic (which many online games are in order to keep players in sync). If you access something outside of the set of synchronized game state, you can cause incredibly painful non-determinism bugs.

  Calls that do modify state, on the other hand, more deeply tie you to those parts of the codebase, and you need to be much more cognizant of that. That makes them good candidates for being rolled up into provided operations in the more visible base class.

- If the implementation of a provided operation only forwards a call to some outside system, then it isn't adding much value. In that case, it may be

simpler to call the outside method directly.

However, even simple forwarding can still be useful — those methods often access state that the base class doesn't want to directly expose to subclasses. For example, let's say `Superpower` provided this:

```
void playSound(SoundId sound, double volume)
{
  soundEngine_.play(sound, volume);
}
```

It's just forwarding the call to some `soundEngine_` field in `Superpower`. The advantage, though, is that it keeps that field encapsulated in `Superpower` so subclasses can't poke at it.

Should methods be provided directly, or through objects that contain them?

The challenge with this pattern is that you can end up with a painfully large number of methods crammed into your base class. You can mitigate that by moving some of those methods over to other classes. The provided operations in the base class then just return one of those objects.

For example, to let a power play sounds, we could add these directly to `Superpower`:

```
class Superpower
{
protected:
  void playSound(SoundId sound, double volume)
  {
    // Code here...
  }

  void stopSound(SoundId sound)
  {
    // Code here...
  }

  void setVolume(SoundId sound)
  {
    // Code here...
  }

  // Sandbox method and other operations...
};
```

But if `Superpower` is already getting large and unwieldy, we might want to avoid that. Instead, we create a `SoundPlayer` class that exposes that functionality:

```
class SoundPlayer
{
  void playSound(SoundId sound, double volume)
  {
    // Code here...
  }

  void stopSound(SoundId sound)
  {
    // Code here...
  }

  void setVolume(SoundId sound)
  {
    // Code here...
  }
};
```

Then `Superpower` provides access to it:

```
class Superpower
{
protected:
  SoundPlayer& getSoundPlayer()
  {
    return soundPlayer_;
  }

  // Sandbox method and other operations...

private:
  SoundPlayer soundPlayer_;
};
```

Shunting provided operations into auxiliary classes like this can do a few things for you:

- *It reduces the number of methods in the base class.* In the example here, we went from three methods to just a single getter.
- *Code in the helper class is usually easier to maintain.* Core base classes like `Superpower`, despite our best intentions, tend to be tricky to change since so much depends on them. By moving functionality over to a less coupled secondary class, we make that code easier to poke at without breaking things.
- *It lowers the coupling between the base class and other systems.* When `playSound()` was a method directly on `Superpower`, our base class was directly tied to `SoundId` and whatever audio code the implementation called into. Moving that over to `SoundPlayer` reduces `Superpower`'s coupling to the single `SoundPlayer` class, which then encapsulates all of its other dependencies.

How does the base class get the state that it needs?

Your base class will often need some data that it wants to encapsulate and keep hidden from its subclasses. In our first example, the `Superpower` class provided a `spawnParticles()` method. If the implementation of that needs some particle system object, how would it get one?

- **Pass it to the base class constructor:**

  The simplest solution is to have the base class take it as a constructor argument:

  ```
  class Superpower
  {
  public:
    Superpower(ParticleSystem* particles)
    : particles_(particles)
    {}

    // Sandbox method and other operations...

  private:
    ParticleSystem* particles_;
  };
  ```

  This safely ensures that every superpower does have a particle system by the time it's constructed. But let's look at a derived class:

  ```
  class SkyLaunch : public Superpower
  {
  public:
    SkyLaunch(ParticleSystem* particles)
  ```

```
  : Superpower(particles)
  {}
};
```

Here we see the problem. Every derived class will need to have a constructor that calls the base class one and passes along that argument. That exposes every derived class to a piece of state that we don't want them to know about.

This is also a maintenance headache. If we later add another piece of state to the base class, every constructor in each of our derived classes will have to be modified to pass it along.

- **Do two-stage initialization:**

  To avoid passing everything through the constructor, we can split initialization into two steps. The constructor will take no parameters and just create the object. Then, we call a separate method defined directly on the base class to pass in the rest of the data that it needs:

```
Superpower* power = new SkyLaunch();
power->init(particles);
```

  Note here that since we aren't passing anything into the constructor for `SkyLaunch`, it isn't coupled to anything we want to keep private in `Superpower`. The trouble with this approach, though, is that you have to make sure you always remember to call `init()`. If you ever forget, you'll have a power that's in some twilight half-created state and won't work.

  You can fix that by encapsulating the entire process into a single function, like so:

```
Superpower* createSkyLaunch(ParticleSystem* particles)
{
  Superpower* power = new SkyLaunch();
  power->init(particles);
  return power;
}
```

  With a little trickery like private constructors and friend classes, you can ensure this `createSkylaunch()` function is the *only* function that can actually create powers. That way, you can't forget any of the initialization stages.

- **Make the state static:**

  In the previous example, we were initializing each `Superpower` *instance* with a particle system. That makes sense when every power needs its own unique state. But let's say that the particle system is a singleton, and every power will be sharing the same state.

  In that case, we can make the state private to the base class and also make it *static*. The game will still have to make sure that it initializes the state, but it only has to initialize the `Superpower` *class* once for the entire game, and not each instance.

  Keep in mind that this still has many of the problems of a singleton. You've got some state shared between lots and lots of objects (all of the `Superpower` instances). The particle system is encapsulated, so it isn't globally *visible*, which is good, but it can still make reasoning about powers harder because they can all poke at the same object.

```
class Superpower
{
public:
  static void init(ParticleSystem* particles)
  {
    particles_ = particles;
  }

  // Sandbox method and other operations...

private:
  static ParticleSystem* particles_;
};
```

Note here that `init()` and `particles_` are both static. As long as the game calls `Superpower::init()` once early on, every power can access the particle system. At the same time, `Superpower` instances can be created freely by calling the right derived class's constructor.

Even better, now that `particles_` is a *static* variable, we don't have to store it for each instance of `Superpower`, so we've made the class use less memory.

- **Use a service locator:**

  The previous option requires that outside code specifically remembers to push in the state that the base class needs before it needs it. That places the burden of initialization on the surrounding code. Another option is to let the base class handle it by pulling in the state it needs. One way to do that is by using the Service Locator → pattern:

```
class Superpower
{
protected:
  void spawnParticles(ParticleType type, int count)
  {
    ParticleSystem& particles = Locator::getParticles();
    particles.spawn(type, count);
  }

  // Sandbox method and other operations...
};
```

  Here, `spawnParticles()` needs a particle system. Instead of being *given* one by outside code, it fetches one itself from the service locator.

See Also

- When you apply the Update Method → pattern, your update method will often also be a sandbox method.
- This pattern is a role reversal of the Template Method GoF pattern. In both patterns, you implement a method using a set of primitive operations. With Subclass Sandbox, the method is in the derived class and the primitive operations are in the base class. With Template Method, the *base* class has the method and the primitive operations are implemented by the *derived* class.
- You can also consider this a variation on the Facade GoF pattern. That pattern hides a number of different systems behind a single simplified API. With Subclass Sandbox, the base class acts as a facade that hides the entire game engine from the subclasses.

© 2009-2014 Robert Nystrom